SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: CSA04 COURSE CODE: Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments. (BL2, BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Total Marks:25**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I M. Bhagyalakshmi (192525439) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Bhagya lakshmi
Signature of the student:

Annexure A

Short Answer Test

Question 1 **10 Marks**

A university computer laboratory allows multiple students to execute programs simultaneously on a shared Linux server.

(a) Identify the type of Operating System suitable for this environment and justify your answer.

(b) State any two responsibilities of the kernel in managing multiple user processes.

(c) Write a C-based system call sequence (or pseudo-code) to create a child process and display the Process IDs of both the parent and child.

Question 2 **10 Marks**

An application needs to store and retrieve student records from a text file.

(a) List the system calls required for file creation, writing, reading, and closing.

(b) Explain the purpose of any two file management system calls.

(c) Write a C-based system call sequence (or pseudo-code) to create a file, write a student name, and close the file.

Question 3 **10 Marks**

A user wants to copy the contents of one file to another.

(a) List the system calls required to complete this operation.

(b) Explain the purpose of any two of these system calls.

(c) Write a pseudo-code or C-based system call sequence to copy the contents of one file to another.

Question 4 **10 Marks**

A program needs to execute another program after creating a child process.

(a) State the purpose of the fork() and exec() system calls.

(b) Explain the role of the wait() system call.

(c) Write a system call sequence using fork(), exec(), and wait().

Question 5 **10 Marks**

An operating system manages multiple files stored on secondary memory.

(a) Explain the importance of file management in an operating system.

(b) State the purpose of the open() and close() system calls.

(c) Write a C-based system call sequence to open a file, read its contents, and close it.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	35%	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25%	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15%	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15%	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10%	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Marks Evaluation:

Question No.	Conceptual Understanding (35%)	Accuracy of Answers (25%)	Application of Knowledge (15%)	Completeness of Response (15%)	Clarity & Presentation (10%)	Total Marks (100%)
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

M. Bhagyalakshmi
192525439

CO1 ASSESSMENT-1

1). a) **Multi-user operating system:-**

A multi-user operating system is suitable for this environment. It allows multiple users to access and use the same computer system simultaneously.

Advantages:-

1. It allows multiple students to use the server at the same time.
2. It manages CPU time among different users and processes.
4. It manages shared resources efficiently.
5. It provides security through user accounts and permissions.

Ex:- Linux and UNIX,

b) **Responsibilities of the kernel**

1. **Process Management:-**

* Creates and terminates processes.
* schedules processors of CPU execution.
* Allocates CPU time to different

processes.

* Manages process states and context switching.

2. Memory Management:-

* Allocates memory to processes.
* Protects one process's memory from another.
* Manages virtual memory.

(1) C program to create a child process

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main()
{
  pid_t pid;
    pid = fork();
    if (pid<0)
    {
      printf("process creation failed\n");
    }
      else if (pid==0)
        printf("child process ID: %d\n");
        getpid();
          printf("child process ID:%d\n",
          getpid());
        }
        else
```

```
{
    printf("parent process ID: %d\n",
        getpid());
    printf("child process ID: %d\n", pid);
}
    return 0;
}
```

Output:-

parent process ID: 1200
child process ID: 1201
child process ID:- 1201
parent process ID:- 1200.

2) a) System calls used in file management:-

1. Open() - opens an existing file or creates a new file with suitable flags.
2. Create() - creates a file or truncates an existing file.
3. Write() - writes data into a file.
4. Read() - Reads data from a file.
5. Close() - closes an opened file.

b) Purpose of Two File Management System Calls:

1. open() system call

The open() system call opens a file for reading or writing. It returns a file description on success.

Syntax:-

```
int fd = open("student.txt", O_WRONLY | O_CREAT, 0644);
```

* student.txt specifies the filename.
* O_WRONLY opens the file for writing.
* O_CREATE creates the file if it does not exist.
* 0644 specifies file permissions.

2. write() system call:-

The write() system call writes data into an opened file.

Syntax:-

```
write(fd, data, length);
```

* fd represents the file description.
* data contains the data to be written.
* length specifies the number of bytes to write.

c) C Program to create a file and write a student name:-

```
#include<stdio.h>
#include<fcntl.h>
#include<unistd.h>
int main()
{
    int fd;
    char name[] = "Hasini";
```

```
fd = open ("student.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);
if (fd == -1)
{
    Printf ("File creation failed.\n");
    return 1;
}
write (fd, name, sizeof (name) - 1);
Close (fd);
Printf ("Student name written successfully\n");
return 0;
}
```

Output:-

Student name written successfully.

File content:-

Hasini

3) a) System Calls Required to copy a File

1. open() - opens the source and destination files.
2. read() - Reads data from the source file.
3. write() - Writes data into the destination file.
4. close() - closes both files after copying.

b)

Purpose of Two System Calls:-

1. read() system call:-

The read() system call reads the data from an opened file into a memory buffer.

Syntax:-

read(fd, buffer, size);

* fd represents the file descriptor.
* buffer stores the data read from the file.
* size specifies the maximum number of bytes to read.

2. write() system call:-

The write() system call writes data from a memory buffer into an opened file.

Syntax:-

write(fd, buffer, size);

* fd represents the destination file descriptor.
* buffer contains the data to write.
* size specifies the number of bytes to write.

c) C Program to copy the contents of one file to another

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
    int source, destination, n;
```

Output:-
File copied successfully.

Result:- The contents of source.txt are copied into destination.txt.

4)

a) Purpose of fork() and exec() system calls

1. fork() system call

The fork() system call creates a new process called a child process by duplicating the calling process.

Purposes:-

* Creates a child process.
* Allows parent and child processes to execute independently.
* Returns 0 to the child process.
* Returns the child's PID to the parent process.
* Returns -1 if process creation fails.

2. exec() system call:-

The exec() family of system calls replaces the current process program with a new program.

Purposes:-

* Executes another program within the current process
* Replaces the existing program's code and memory image.
* Does not create a new process by itself.
* Does not return to the original program

```
char buffer[100];

source = open ("source.txt", O_RDONLY);
if (source == -1)
{
  Printf ("cannot open source file\n");
}

destination= open ("destination.txt",
                   O_RDONLY | O_CREAT |
                   O_TRUNC, 0644);
if(destination == -1)
{
   Printf ("cannot open destination file\n");
    close (source);
     return 1;
}
while ((n = read (source, buffer, sizeof (buffer))
      > 0)
{
   write (destination, buffer, n);
}
 close (source);
 close (destination);
  Printf ("File copied successfully\n");
    return 0;
}
```

when successful.

b) Role of the wait() system call:

The wait() system call allows a parent process to wait until one of the child processes terminates.

Functions:

1. Suspends the parent process until a child terminates or the specified wait condition is met.
2. Allows the parent to collect the child's termination status.
3. Coordinates the execution of parent and child processes.

Syntax:

```
wait (NULL);
```

c) C Program using fork(), exec(), and wait()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main()
{
    pid_t pid;
    pid = fork();
    if (pid < 0)
    printf("Fork failed\n");
        return 1;
    }
    else if (pid == 0)
```

```
{
    printf("child process executing\n");
    execlp("ls", "ls", "-l", (char*) NULL);
    printf("ls", "ls", "-l", (char*) NULL);
    Printf("Exec. failed\n");
    return 1;
}
else
{
    wait(NULL);
    Printf("Child process completed\n");
}
return 0;
}
```

Output:-

```
Child process executing.
total 8.
-rw -r--r-- 1 usq 120 file1.txt.
-rw-r-- r--1 usq usq 120 file2.txt.
Child process completed.
```

Result:- The child process executes the ls-l command, and the parent waits until the child process terminates.

5) a) Importance of File Management in an operating System:-

File management is an important function of an operating system that organizes, stores, retrieves, and protects files on secondary storage devices.

Importance:-

1. File organization:- Organizes files into directories and folders for easy access.
2. File creation & Deletion:- Allows users to create, rename, and delete files.
3. Storage Management:- Tracks file locations and available disk space.
4. Access control:- Protects files through permissions and access restrictions.
5. Data Retrieval:- Allows users and applications to retrieve stored information.
6. File Sharing:- Supports controlled sharing of files between users and programs.
7. Data Integrity:- Helps maintain the consistency of stored data through proper file operations.

b) Purpose of open() and close() System calls:-

1. open() System call:-

The open() system call opens an existing file or creates a new file when used with appropriate flags.

Syntax:-

```
fd = open ("data.txt", O_RDONLY);
```

Purposes:-

* Opens a file for reading or writing.
* Returns a file descriptor on success.
* Allows a program to access the file contents.

2. close() system call:-

The close() system call closes an opened file descriptor.

Syntax:-

```
close (fd);
```

Purposes:-

* Release the file descriptor.
* Helps free operating system resources associated with the open file.

(c) Code:-

```
#include <fcntl.h>
#include <unistd.h>
int main()
{
    Char b[100];
    int f = open("student.txt", O_RDONLY);
        int n = read (f, b, 99);
            b[n] = '\0';
            printf ("%s", b);
            close (f);
}
```

Result:- The program opens the file, reads and displays its contents, and closes the file.